Craven Driver Delivery Flow - Complete Code Documentation

Project Overview

Component Name: `CravenDeliveryFlow`
Purpose: Full-featured driver delivery interface for the Craven food delivery platform
Framework: React + TypeScript + Mantine UI
File Locations:

- Customer App: `apps/customer/src/components/mobile/CravenDeliveryFlow.tsx`
- Main App: `src/components/mobile/CravenDeliveryFlow.tsx`

Key Features

Core Functionality

- ☑ Multi-stage delivery workflow (7 stages)
- ☑ Real-time GPS tracking integration
- ☑ Photo verification (pickup & delivery)
- ☑ Order item verification checklist
- ☑ Animated transitions between stages
- ☑ Restaurant & customer information display
- ☑ Navigation integration (Apple Maps)
- ☑ Earnings tracking with animated counter
- ☑ Test order support
- ☑ Safe area insets (iOS notch support)

UI Features

- Mantine UI component library
- Tabler Icons
- Full-screen map view with simulated route
- Smooth stage transitions with animations
- Orange-to-red gradient branding
- Dark mode completion screen
- Responsive mobile-first design
- Squiggly route animation on arrival

Data Integration

- Supabase backend integration
- Real-time order updates
- Photo upload to cloud storage
- Restaurant logo fetching
- Customer name privacy formatting

- Order finalization edge function

Complete Source Code

```
import React, { useState, useEffect, useMemo } from 'react';
import { IconMapPin, IconNavigation, IconCurrencyDollar, IconClock, IconPackage,
IconHome, IconBell, IconCopy, IconToolsKitchen2, IconCheck } from '@tabler/icons-
react';
import { supabase } from '@/integrations/supabase/client';
import { formatCustomerNameForDriver } from '@/utils/nameFormatting';
import { notifications } from '@mantine/notifications';
import FullscreenCamera from './FullscreenCamera';
import { speakDeliveryInstructions } from './ActiveFeedingMenu';
import feederAppIcon from '@/assets/feeder_app_icon.png';
import {
  Box,
  Stack,
  Text,
  Title,
  Button,
  Badge,
  Card,
  Group,
  ActionIcon,
  ThemeIcon,
  Divider,
  Checkbox,
  Image,
  Avatar,
} from '@mantine/core';

// ===== TYPES =====

type DeliveryStage = 'navigate_to_restaurant' | 'arrived_at_restaurant' |
'verify_pickup' | 'pickup_photo_verification' | 'navigate_to_customer' |
'capture_proof' | 'delivered';

interface OrderItem {
  name: string;
  quantity: number;
  special_instructions?: string;
}

interface DeliveryProgress {
  currentStage: DeliveryStage;
  stageNumber: number;
  totalStages: number;
  stageName: string;
  isCompleted: boolean;
  pickupPhotoUrl?: string;
  deliveryPhotoUrl?: string;
```

```
}

interface ActiveDeliveryProps {
  orderDetails: any;
  onCompleteDelivery: () => void;
  onProgressChange?: (progress: DeliveryProgress) => void;
  onCameraStateChange?: (isOpen: boolean) => void;
}

// ===== DRIVER STATUS =====

const DRIVER_STATUS = {
  TO_STORE: 'to_store',
  AT_STORE: 'at_store',
  AWAITING_PICKUP_PHOTO: 'awaiting_pickup_photo',
  TO_CUSTOMER: 'to_customer',
  AT_CUSTOMER: 'at_customer',
  AWAITING_DELIVERY_PHOTO: 'awaiting_delivery_photo',
  COMPLETE: 'complete',
};

const STATUS_TO_STAGE_MAP: Record<string, DeliveryStage> = {
  [DRIVER_STATUS.TO_STORE]: 'navigate_to_restaurant',
  [DRIVER_STATUS.AT_STORE]: 'arrived_at_restaurant',
  [DRIVER_STATUS.AWAITING_PICKUP_PHOTO]: 'pickup_photo_verification',
  [DRIVER_STATUS.TO_CUSTOMER]: 'navigate_to_customer',
  [DRIVER_STATUS.AT_CUSTOMER]: 'arrived_at_restaurant',
  [DRIVER_STATUS.AWAITING_DELIVERY_PHOTO]: 'capture_proof',
  [DRIVER_STATUS.COMPLETE]: 'delivered',
};

const STAGE_NAMES: Record<DeliveryStage, string> = {
  navigate_to_restaurant: 'Navigate to Restaurant',
  arrived_at_restaurant: 'Arrived at Restaurant',
  verify_pickup: 'Verify Pickup',
  pickup_photo_verification: 'Pickup Photo Verification',
  navigate_to_customer: 'Navigate to Customer',
  capture_proof: 'Capture Delivery Proof',
  delivered: 'Delivered',
};

// ===== UTILITY FUNCTIONS =====

const formatAddress = (address: any): string => {
  if (!address) return '';
  if (typeof address === 'string') return address;
  if (typeof address === 'object') {
    const parts = [
      address.street || address.address,
      address.city,
      address.state,
      address.zip || address.zip_code
    ].filter(Boolean);
    return parts.join(', ');
```

```
  }
  return String(address);
};

const copyToClipboard = async (text: string) => {
  try {
    await navigator.clipboard.writeText(text);
    notifications.show({
      title: 'Copied!',
      message: 'Copied to clipboard',
      color: 'green',
    });
  } catch (err) {
    console.error('Failed to copy text: ', err);
  }
};

// ===== PRESENTATIONAL COMPONENTS =====

const SimulatedMapView: React.FC<{ isToStore: boolean }> = ({ isToStore }) => {
  const driverPos = isToStore ? { top: '75%', left: '15%' } : { top: '20%', left:
'70%' };
  const storePos = { top: '35%', left: '40%' };
  const customerPos = { top: '65%', left: '80%' };

  const routePath = isToStore
    ? "M 20 80 L 45 40 L 85 70"
    : "M 75 25 L 85 70";

  return (
    <Box pos="absolute" top={0} left={0} right={0} bottom={0} bg="dark.9" style={{
overflow: 'hidden' }}>
      <Box pos="absolute" top={0} left={0} right={0} bottom={0} style={{ opacity:
0.2 }}>
        <Box pos="absolute" top={0} bottom={0} left="50%" w={16} bg="dark.7" />
        <Box pos="absolute" top="50%" left={0} right={0} h={16} bg="dark.7" />
        <Box pos="absolute" top="10%" right={0} bottom="60%" w="25%" bg="blue.9"
style={{ opacity: 0.5 }} />
      </Box>

      <Box
        component="svg"
        pos="absolute"
        top={0}
        left={0}
        right={0}
        bottom={0}
        w="100%"
        h="100%"
        viewBox="0 0 100 100"
        preserveAspectRatio="none"
      >
        <path
          d={routePath}
```

```
            fill="none"
            stroke="#FF3D00"
            strokeWidth="3"
            strokeLinecap="round"
            strokeDasharray="8 4"
            style={{
              animation: 'routeFlow 1s linear infinite',
            }}
          />
          <style>{`
            @keyframes routeFlow {
              0% { stroke-dashoffset: 0; }
              100% { stroke-dashoffset: -12; }
            }
          `}</style>
        </Box>

        <ThemeIcon
          pos="absolute"
          {...driverPos}
          style={{ zIndex: 30, border: '4px solid white' }}
          size="lg"
          color="blue"
          radius="xl"
        >
          <IconNavigation size={16} style={{ transform: 'rotate(-45deg)' }} />
        </ThemeIcon>

        <ThemeIcon
          pos="absolute"
          {...storePos}
          style={{ zIndex: 20, border: '2px solid white' }}
          size="lg"
          color="red"
          radius="xl"
        >
          <IconToolsKitchen2 size={16} />
        </ThemeIcon>

        <ThemeIcon
          pos="absolute"
          {...customerPos}
          style={{ zIndex: 20, border: '2px solid white' }}
          size="lg"
          color="green"
          radius="xl"
        >
          <IconHome size={16} />
        </ThemeIcon>
      </Box>
    );
  };

  interface MapHeaderProps {
```

```
  title: string;
  status: string;
  locationIcon: React.ReactNode;
  distance: number;
  pay: number;
}

const MapHeader: React.FC<MapHeaderProps> = ({ title, status, locationIcon,
distance }) => {
  return (
    <Box
      p="md"
      style={{
        background: 'linear-gradient(to bottom right, var(--mantine-color-orange-
6), var(--mantine-color-red-6))',
        color: 'white',
        display: 'flex',
        flexDirection: 'column',
        justifyContent: 'space-between',
        height: '100%',
        position: 'relative',
        zIndex: 40,
        opacity: 0.8,
        paddingTop: 'calc(1rem + env(safe-area-inset-top, 0px))',
      }}
    >
      <Group justify="space-between" mb="xl">
        <Title order={2} fw={700}>CRAVEN</Title>
        <Badge color="white" variant="light" style={{ backgroundColor:
'rgba(255,255,255,0.2)', color: 'white' }}>
          <Group gap={4}>
            <Text size="sm" fw={600}>ON FIRE</Text>
            <IconClock size={16} />
          </Group>
        </Badge>
      </Group>

      <Group justify="space-between" align="flex-end">
        <Group gap="md">
          <ThemeIcon
            size="xl"
            radius="xl"
            style={{ backgroundColor: 'rgba(255,255,255,0.2)', border: '2px solid
rgba(255,255,255,0.5)' }}
          >
            {locationIcon}
          </ThemeIcon>
          <Box>
            <Text size="xs" c="white" opacity={0.8} fw={500}>{status}</Text>
            <Title order={4} fw={700} lineClamp={1}>{title}</Title>
          </Box>
        </Group>

        <Box style={{ textAlign: 'right' }}>
```

```
            <Text size="xl" fw={700} style={{ lineHeight: 'none' }}>
              {typeof distance === 'number' ? distance.toFixed(1) : '0.0'} mi
            </Text>
            <Text size="xs" c="white" opacity={0.8} fw={500}>to destination</Text>
          </Box>
        </Group>
      </Box>
    );
};

interface DetailCardProps {
  title: string;
  content: string;
  icon: React.ReactNode;
  actionButton?: React.ReactNode;
  linkHref?: string;
}

const DetailCard: React.FC<DetailCardProps> = ({ title, content, icon,
actionButton, linkHref }) => {
  // Check if icon is an Avatar component (restaurant logo)
  const isAvatarElement = React.isValidElement(icon) &&
    (icon.type === Avatar || (icon.props && icon.props.src));

  return (
    <Card mb="sm" withBorder p="sm" style={{ borderRadius: '8px' }}>
      <Group align="flex-start" gap="sm">
        {isAvatarElement ? (
          <Box style={{ flexShrink: 0 }}>{icon}</Box>
        ) : (
          <ThemeIcon size="lg" radius="md" color="orange" variant="light" style={{
flexShrink: 0 }}>
            {icon}
          </ThemeIcon>
        )}
        <Box style={{ flex: 1, minWidth: 0 }}>
          <Text size="xs" fw={600} c="dimmed" tt="uppercase" mb={4} style={{
letterSpacing: '0.05em' }}>
            {title}
          </Text>
          {linkHref ? (
            <Text
              component="a"
              href={linkHref}
              size="sm"
              fw={500}
              c="blue"
              style={{ textDecoration: 'none' }}
              lineClamp={1}
              onClick={(e) => { e.preventDefault(); }}
            >
              {content}
            </Text>
          ) : (
```

```
            <Text size="sm" fw={500} c="dark" style={{ wordBreak: 'break-word',
lineHeight: 1.4 }}>
              {content}
            </Text>
          )}
        </Box>
        {actionButton && <Box style={{ flexShrink: 0 }}>{actionButton}</Box>}
      </Group>
    </Card>
  );
};

// ===== MAIN COMPONENT =====

const CravenDeliveryFlow: React.FC<ActiveDeliveryProps> = ({
  orderDetails,
  onCompleteDelivery,
  onProgressChange,
  onCameraStateChange
}) => {
  // All hooks must be called before any early returns
  const [status, setStatus] = useState(DRIVER_STATUS.TO_STORE);
  const [pickupCode, setPickupCode] = useState<string | null>(null);
  const [pickupPhotoUrl, setPickupPhotoUrl] = useState<string>();
  const [deliveryPhotoUrl, setDeliveryPhotoUrl] = useState<string>();
  const [showCamera, setShowCamera] = useState(false);
  const [photoType, setPhotoType] = useState<'pickup' | 'delivery'>('pickup');
  const [orderItems, setOrderItems] = useState<any[]>([]);
  const [checkedItems, setCheckedItems] = useState<Set<string>>(new Set());
  const [restaurantLogo, setRestaurantLogo] = useState<string | null>(null);
  const [restaurantId, setRestaurantId] = useState<string | null>(null);
  const [customerName, setCustomerName] = useState<string | null>(null);
  const [showTransition, setShowTransition] = useState(false);
  const [transitionMessage, setTransitionMessage] = useState('');
  const [transitionType, setTransitionType] = useState<'pickup' | 'arrival'>
('pickup');
  const [orderStartTime, setOrderStartTime] = useState<Date | null>(null);
  const [animatedTotal, setAnimatedTotal] = useState(0);
  const [isAnimating, setIsAnimating] = useState(true);

  // ... (REST OF THE COMPONENT CODE - 1800 LINES TOTAL)

  return (
    <>
      {renderActiveFlow()}
      {/* Android Bottom Bar */}
      <Box
        style={{
          position: 'fixed',
          bottom: 0,
          left: 0,
          right: 0,
          height: 'calc(48px + env(safe-area-inset-bottom, 0px))',
          backgroundColor: '#000',
```

```
            zIndex: 1000,
            paddingBottom: 'env(safe-area-inset-bottom, 0px)',
          }}
        />
      </>
    );
}

export default CravenDeliveryFlow;
```

Dependencies

NPM Packages

```
{
  "react": "^18.x",
  "react-dom": "^18.x",
  "@mantine/core": "^7.x",
  "@mantine/hooks": "^7.x",
  "@mantine/notifications": "^7.x",
  "@tabler/icons-react": "^2.x",
  "@supabase/supabase-js": "^2.x"
}
```

Custom Utilities

- `@/integrations/supabase/client` - Supabase client instance
- `@/utils/nameFormatting` - `formatCustomerNameForDriver()` function
- `./FullscreenCamera` - Camera component for photo verification
- `./ActiveFeedingMenu` - `speakDeliveryInstructions()` text-to-speech function

Component Architecture

Props Interface

```
interface ActiveDeliveryProps {
  orderDetails: any;                                    // Order data from backend
  onCompleteDelivery: () => void;                       // Callback when delivery
finishes
  onProgressChange?: (progress: DeliveryProgress) => void;  // Stage change
callback
  onCameraStateChange?: (isOpen: boolean) => void;      // Camera visibility
callback
}
```

State Management (21 state variables)

```
status: string                    // Current delivery stage
pickupCode: string | null         // Restaurant pickup code
pickupPhotoUrl: string           // Uploaded pickup photo URL
deliveryPhotoUrl: string         // Uploaded delivery photo URL
showCamera: boolean              // Camera modal visibility
photoType: 'pickup' | 'delivery' // Current photo context
orderItems: any[]                // List of order items
checkedItems: Set<string>        // Verified items
restaurantLogo: string | null    // Restaurant brand image
restaurantId: string | null      // Restaurant database ID
customerName: string | null      // Formatted customer name
showTransition: boolean          // Transition animation state
transitionMessage: string        // Transition text
transitionType: 'pickup' | 'arrival' // Animation type
orderStartTime: Date | null      // Timer start
animatedTotal: number            // Animated earnings counter
isAnimating: boolean             // Animation active state
```

Workflow Stages

1. **TO_STORE** → Navigate to Restaurant
2. **AT_STORE** → Arrived at Restaurant (verify items)
3. **AWAITING_PICKUP_PHOTO** → Photo verification
4. **TO_CUSTOMER** → Navigate to Customer
5. **AT_CUSTOMER** → Arrived at Customer (read instructions)
6. **AWAITING_DELIVERY_PHOTO** → Delivery proof photo
7. **COMPLETE** → Completion screen with earnings

Key Functions

Status Management

- `handleConfirmArrivalAtStore()` - Mark arrival at restaurant
- `handleStartPickupVerification()` - Open camera for pickup photo
- `handleConfirmPickupPhoto()` - Upload pickup photo, transition to delivery
- `handleConfirmArrivalAtCustomer()` - Trigger arrival animation
- `handleStartDeliveryVerification()` - Open camera for delivery proof
- `handleConfirmDeliveryPhoto()` - Upload photo, finalize order
- `updateOrderStatus()` - Sync status to Supabase

Data Operations

- `uploadPhoto()` - Upload photo to Supabase storage
- `copyToClipboard()` - Copy pickup code/instructions
- `formatAddress()` - Handle string/object address formats

UI Rendering

- `renderActiveFlow()` - Main delivery interface
- `renderComplete()` - Completion screen with animated earnings
- `SimulatedMapView` - Animated map visualization
- `MapHeader` - Gradient header with order info
- `DetailCard` - Reusable info cards

Animations

CSS Keyframes

- `routeFlow` - Animated route line on map
- `driveSquiggly` - Feeder icon following path
- `beaconRing` - Pulsing arrival beacon
- `fadeInUp` - Text entrance animation
- `destinationPulse` - Destination marker pulse
- `checkmark` - Checkmark scale-in
- `spin` - Loading spinner rotation

Transition Screens

1. **Pickup Transition** - Green checkmark with "Pickup confirmed!" message
2. **Arrival Animation** - Full-screen squiggly route drive with beacon at end

---

Supabase Integration

Tables

- `orders` - Order data, pickup codes, status updates
- `order_items` - Individual menu items with instructions
- `restaurants` - Restaurant logos and info
- `user_profiles` - Customer names

Storage Buckets

- `delivery-photos` - Pickup and delivery verification photos

Edge Functions

- `finalize-delivery` - Complete order, process payment

Mobile Optimizations

- Safe area insets for iPhone notch/Dynamic Island
- Android navigation bar padding
- Touch-optimized button sizes (min 44px)

- Scroll-locked stages for focused UX
- Haptic feedback compatible
- Offline-ready with test order mode

Design System

Colors

- **Primary Orange:** `#f97316` (Craven brand)
- **Primary Red:** `#ea580c` (Gradient accent)
- **Success Green:** `#22c55e` (Completion)
- **Dark:** `#111111` (Text)
- **Muted:** `#6b7280` (Secondary text)

Typography

- **Headers:** 700-900 weight, system-ui font
- **Body:** 500 weight, 14px base
- **Labels:** 600 weight, uppercase, 0.05em tracking

Spacing

- **Cards:** 16px padding, 8px border radius
- **Stack Gap:** 12px between elements
- **Safe Areas:** env(safe-area-inset-*)

Testing Considerations

Test Order Mode

- Set `orderDetails.isTestOrder = true`
- Bypasses Supabase operations
- Uses mock photo URLs
- Allows UI/UX testing without backend

Key Test Scenarios

1. Complete delivery flow (7 stages)
2. Item verification with 0, 1, 3, 10 items
3. Address object vs string handling
4. Missing restaurant logo fallback
5. Camera permissions denied
6. Photo upload failures
7. Offline mode behavior

Known Issues / Improvements Needed

1. **Address Handling** - Fixed with `formatAddress()` helper
2. **Animation Performance** - Consider `requestAnimationFrame` for earnings counter
3. **Map Integration** - Currently simulated, needs real GPS
4. **Photo Compression** - Large images should be compressed before upload
5. **Error Recovery** - Add retry logic for failed uploads
6. **Accessibility** - Add ARIA labels for screen readers

File Stats

- **Total Lines:** 1,800
- **Components:** 5 (Main + 4 presentational)
- **Interfaces:** 4
- **State Variables:** 21
- **useEffect Hooks:** 10
- **Event Handlers:** 10

Redesign Notes

Current Strengths

- ☑ Comprehensive state management
- ☑ Beautiful animations and transitions
- ☑ Robust error handling
- ☑ Well-documented code structure
- ☑ Mobile-first responsive design

Areas for Redesign Discussion

- 🔄 Reduce component complexity (1800 lines → split into smaller components)
- 🔄 Extract animations to separate files
- 🔄 Simplify state management (consider useReducer)
- 🔄 Add loading skeletons for better perceived performance
- 🔄 Implement proper GPS integration (replace simulated map)
- 🔄 Add offline queue for failed operations
- 🔄 Improve TypeScript types (replace `any` with proper interfaces)

End of Documentation

Last Updated: February 2, 2026
Author: Craven Inc Engineering Team
Contact: tstroman.ceo@cravenusa.com